

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

FastSLAM: Simultaneous Localization and Mapping with Rao-Blackwellized Particle Filters

Jan 07, 2026

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt

AIMotion, AI and Sensor Data Fusion

Email: bjoern.giesler@thi.de

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **News on Moodle:**
  - Slides for the lecture (upcoming lectures marked as WIP and are subject to change!)
  - Formulary, Exercises (Solutions provided after discussion in the lecture)
  - Exercises are optional, one task / sheet per week, but are highly recommended.
    They can be handed in for correction.
  - I will *try* to publish exercises before Wednesday‘s lecture so that initial questions can be discussed on Thursday.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

- **Previous lectures:**
  - Mathematical details of Particle Filters
  - Joint Probability Data Association Filters
  - JPDAs and Particle Filters
  - Simultaneous Localization and Mapping (based on Kalman Filters)
- **This week:**
  - Simultaneous Localization and Mapping (based on Particle Filters)
  - Overview over features for SLAM
  - Occupancy Grids and Grid-based SLAM
- **Upcoming:**
  - (DNN-based) early fusion approaches.

Let's talk exam!

- **I want to know what's in your brain! My priority is always**
  1. have you understood the concept?
  2. ...thoroughly enough so you could implement it practically?
  3. and last, are the numerical results you obtain correct?
- **No formulary, personal notes, tablets, laptops, AI assistants or personal humanoid messenger or telepresence robots allowed.**
- **Pocket calculators are a necessity.**
- **There will likely be some hand calculation with the help of pocket calculators.**
- **No coding of course, but questions about algorithms in pseudo code are possible.**
- **Long answer questions possible. Multiple choice questions possible. Formula questions possible.**
- **Basically everything we discussed in the lecture is relevant. Do not rely on only the exercise sheets or programming exercises to prepare.**
- **I will be fair!**

Exercise 6 – Solution

EKF Correction (thank you Danil Afanasyev!)

- Predict

$$\hat{\mu}_{t+1|t} = f(\hat{\mu}_{t|t}, \mathbf{u}_t)$$
$$\Sigma_{t+1|t} = F\Sigma_{t|t}F^\top + Q$$

- Measure

$$\hat{\mathbf{z}}_{t+1} = H\hat{\mu}_{t+1|t}$$
$$\mathbf{z}_{t+1} = h(\mu_{t+1})$$

- Kalman Gain

$$K_t = \Sigma_{t|t-1}H^\top(H\Sigma_{t|t-1}H^\top + R)^{-1}$$

- Update Estimate

~~$\hat{\mu}_{t+1|t+1} = \hat{\mu}_{t+1|t} + K_t \cdot (\mathbf{z}_{t+1} - H\hat{\mu}_{t+1|t})$~~

$$\hat{\mu}_{t+1|t+1} = \hat{\mu}_{t+1|t} + K_t \cdot (\mathbf{z}_{t+1} - h(\hat{\mu}_{t+1|t}))$$

- Update Covariance

$$\Sigma_{t+1|t+1} = \Sigma_{t+1|t} - K_t H \Sigma_{t+1|t}$$

Correction
Both variants are valid, but this is more precise

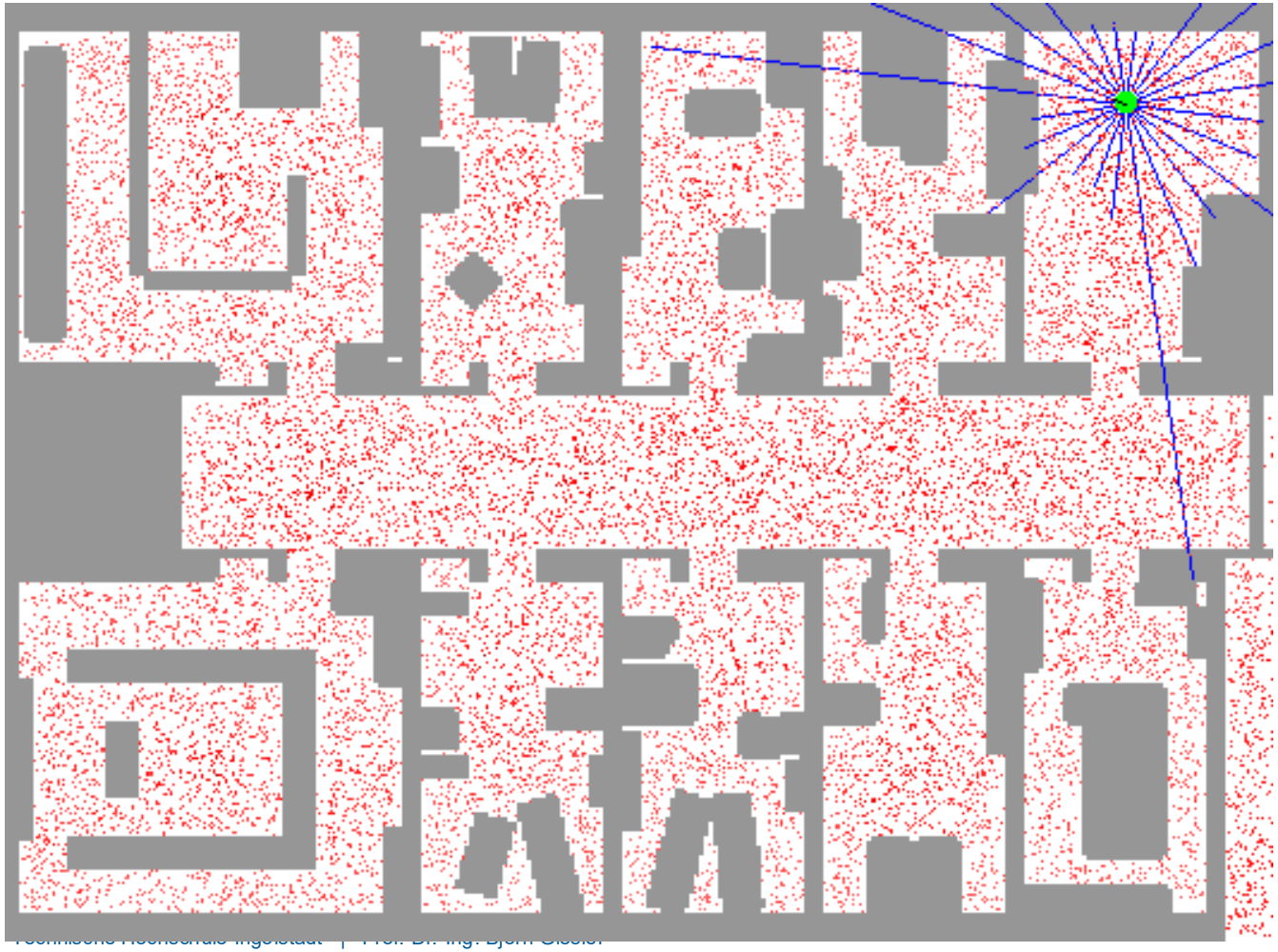
with F the Jacobian of f and H the Jacobian of h.

SLAM and Particle Filters

EKF-SLAM Recap

- **We have seen that the Extended Kalman Filter provides a strong framework for localization and mapping, *provided the correspondences between observed and mapped features are correct*.**
- **If feature correspondences are observed, the filter always converges.**
- **If the observed correspondences are not correct (i.e. we are making a mistake in matching observed features to mapped features), it converges to a wrong map.**
- **This especially becomes catastrophic when loop closing:**
  - Wrong feature correspondences lead to wrong localization...
  - ...but also to an increased confidence in that localization...
  - ...which in turn maps the localization error to all previously mapped features...
  - ...but also increases the confidence in the map.
- **Solutions to this problem:**
  - Decrease the likelihood of matching errors by using stronger features (will be discussed later)
  - Decrease the impact of correspondence errors on the map.

Recap: Monte Carlo Localization



- We have seen that **EKF-SLAM** provides

Particle Filter Recap

- **Non-parametric, recursive Bayes filter**
- **Posterior is represented by a set of weighted samples**
- **Not limited to Gaussians**
- **Proposal distribution used to draw new samples**
- **Weighting samples to account for the differences between the proposal and the target distributions**
- **Works well in low-dimensional spaces**
- **Algorithm:**

1. Sample next particle set using proposal distribution $x_t^i \sim \text{proposal}(x_t | \dots)$
2. Compute importance weights $w_t^i = \frac{\text{target}(x_t^i)}{\text{proposal}(x_t^i)}$
3. Resample: "Replace unlikely samples by more likely ones"

Particle Representation

- **A set of weighted samples** $\mathcal{X} = \{(\mathbf{x}^i, w^i)\}_{i=1,\dots,N}$
- **Think of a sample as one hypothesis about the state.**
- **For feature-based SLAM:** $\mathbf{x} = (\underbrace{x_{1:t}, y_{1:t}, \theta_{1:t}}_{\text{pose}}, \underbrace{l_{1,x}, l_{1,y}, \dots, l_{M,x}, l_{M,y}}_{\text{landmarks}})^\top$

Particle Representation

- **A set of weighted samples** $\mathcal{X} = \{(\mathbf{x}^i, w^i)\}_{i=1,\dots,N}$
- **Think of a sample as one hypothesis about the state.**
- **For feature-based SLAM:** $\mathbf{x} = (\underbrace{x_{1:t}, y_{1:t}, \theta_{1:t}}_{\text{pose}}, \underbrace{l_{1,x}, l_{1,y}, \dots, l_{M,x}, l_{M,y}}_{\text{landmarks}})^\top$
- **The problem with this is the dimensionality.**
  - Particle filters need reasonably dense coverage of the state space with samples.
  - Therefore they are effective in low-dimensional spaces.
  - Covering the full state space of $\mathbf{x} = (x_{1:t}, y_{1:t}, \theta_{1:t}, l_{1,x}, l_{1,y}, \dots, l_{M,x}, l_{M,y})^\top$ densely with samples is just too expensive.
  - It also does not deal well with map management (adding landmarks as they emerge, removing them as they become invalid or irrelevant) since that would require touching every sample every time.
- **How can we simplify the problem?**

Can we exploit or decouple dependencies?

- **If we know the poses of the robot, mapping is easy.**

$$\mathbf{x} = (x_{1:t}, y_{1:t}, \theta_{1:t}, l_{1,x}, l_{1,y}, \ldots, l_{M,x}, l_{M,y})^\top$$



- **Key idea:**
  - Use the particle set only to model the robot's path.
  - Each sample is a path hypothesis.
  - For each sample, we can then compute an individual map of landmarks.
  - The map depends on the poses of the robot.
  - We know how to build a map if the position of the sensor is known.
  - Given known robot pose, we can estimate locations of all features independent of each other!

Factored Posterior Landmarks

- **Notation: Instead of writing** $(x_{1:t}, y_{1:t}, \theta_{1:t}, l_{1,x}, l_{1,y}, \ldots, l_{M,x}, l_{M,y})^\top$,

  we will use the shorter $(\mathbf{x}_{1:t}, \mathbf{m}_{1:M})^\top$ **for the robot pose and the map.**

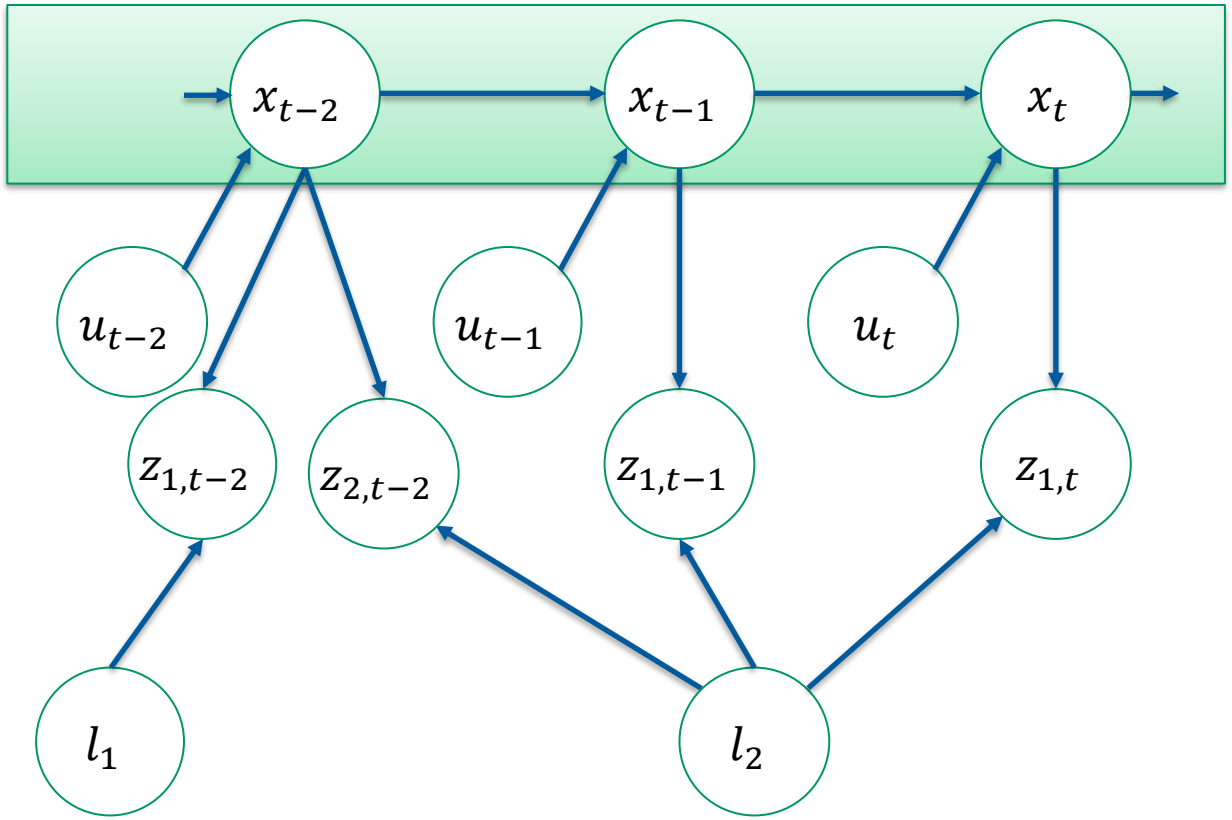
- **Bayesian representation:**

poses map observations movements

$$\underbrace{p(\mathbf{x}_{1:t}, \mathbf{m}_{1:M} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t})}_{\text{SLAM posterior}} = \underbrace{p(\mathbf{x}_{1:t} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t})}_{\text{Robot path posterior}} \underbrace{p(\mathbf{m}_{1:M} | \mathbf{x}_{1:t}, \mathbf{z}_{1:t})}_{\text{Map posterior}}$$

- **Does knowing the robot path help to solve the problem?**

Graphical Model



Assumed known

Observations

Landmarks

Landmark positions and observations are coupled through the robot positions.

If we assume the positions as known, the landmarks become conditionally independent.

Picture: [1]

Factored Posterior

- **Using the assumption that all map features are independent, we can factorize the posterior:**

$$p(\mathbf{x}_{1:t}, \mathbf{m}_{1:M} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) = p(\mathbf{x}_{1:t} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) p(\mathbf{m}_{1:M} | \mathbf{x}_{1:t}, \mathbf{z}_{1:t})$$

$$= \underbrace{p(\mathbf{x}_{1:t} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t})}_{\text{Robot path posterior}} \underbrace{\prod_{i=1}^{M} p(l_i | \mathbf{x}_{1:t}, \mathbf{z}_{1:t})}_{\text{Landmark positions}}$$

Localization Problem

Mapping Problem

Factored Posterior

- **Using the assumption that all map features are independent, we can factorize the posterior:**

$$p(\mathbf{x}_{1:t}, \mathbf{m}_{1:M} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) = p(\mathbf{x}_{1:t} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) p(\mathbf{m}_{1:M} | \mathbf{x}_{1:t}, \mathbf{z}_{1:t})$$

$$= \underbrace{p(\mathbf{x}_{1:t} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t})}_{\text{Robot path posterior}} \underbrace{\prod_{i=1}^{M} p(l_i | \mathbf{x}_{1:t}, \mathbf{z}_{1:t})}_{\text{Landmark positions}}$$

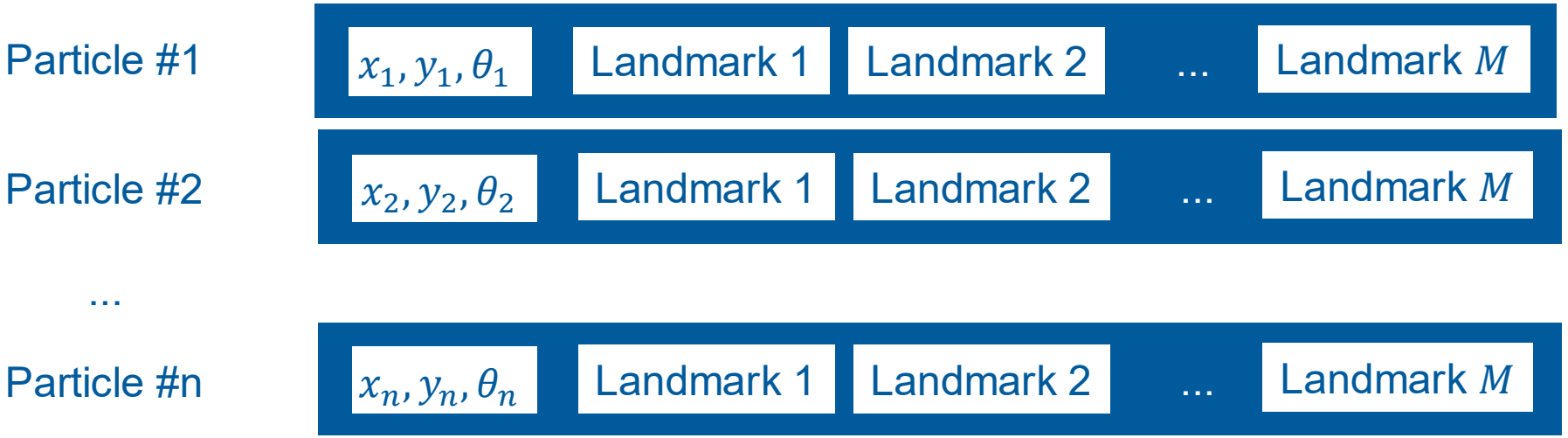
Localization Problem

Mapping Problem

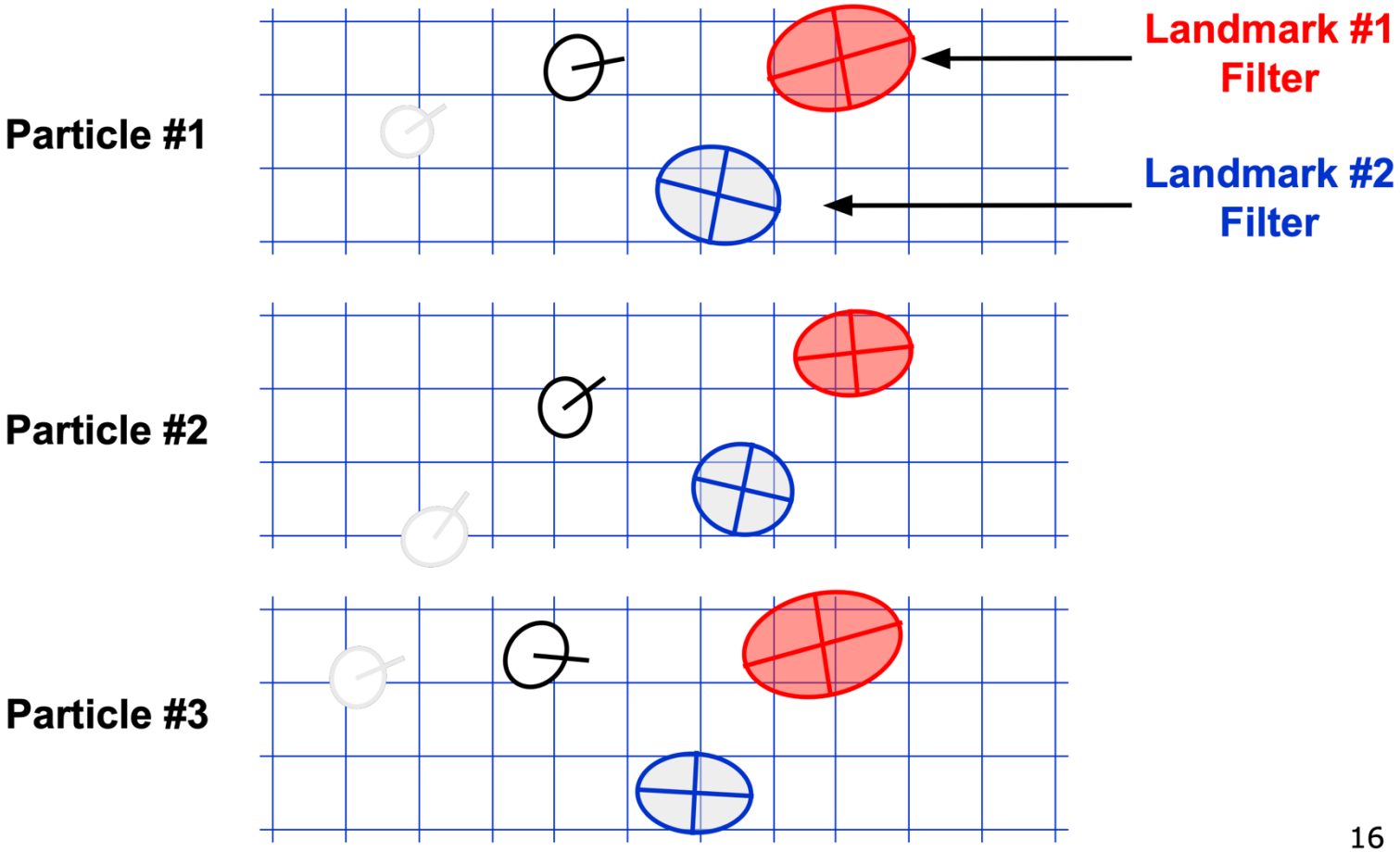
- **This factorization is called *Rao-Blackwellization.* (Explanation and derivation in [1].)**

Rao-Blackwellized Particle Filter – The FastSLAM Algorithm

- **FastSLAM [2] is a SLAM algorithm using Rao-Blackwellized Particle Filter / Kalman Filter combination.**
  1. estimate the robot pose using a particle filter (solving the localization problem),
  2. associate each particle with a map represented by a set of Gaussian-distributed landmark positions (solving the mapping problem),
  3. estimate the landmark positions using separate EKFs,
  4. compute particle weights based on correspondence between landmarks and observations (e.g. with Mahalanobis).

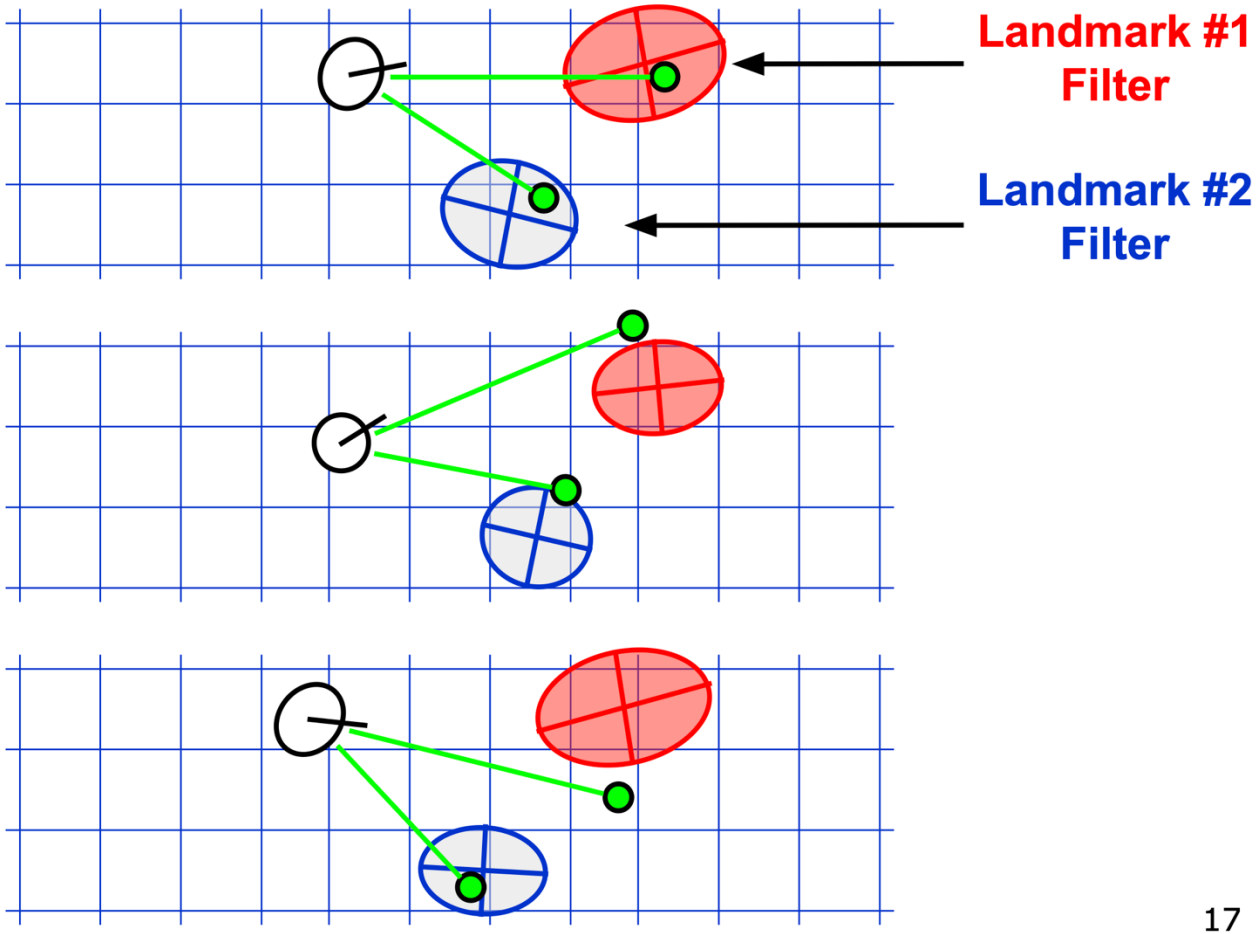


FastSLAM – Action Update



16

FastSLAM – Sensor Update



Particle weight based on observation / landmark correspondence

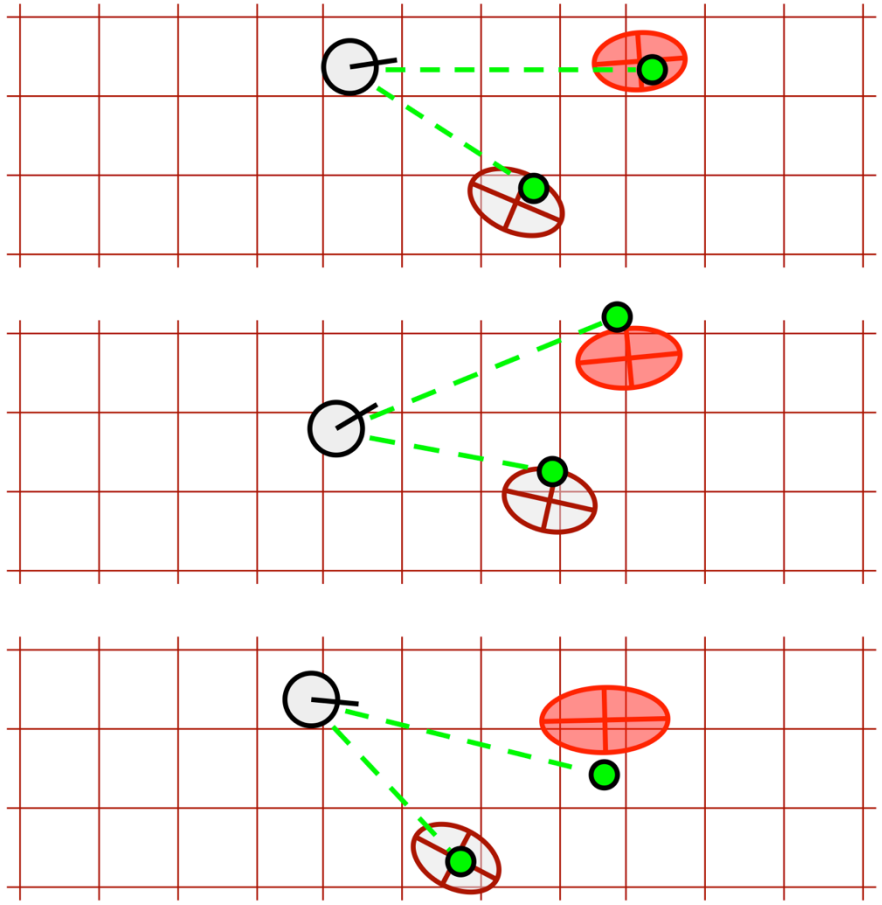
Weight = 0.6

Weight = 0.3

Weight = 0.1

17

FastSLAM – Map Update



Particle weight based on observation / landmark correspondence

Weight = 0.6

Weight = 0.3

Weight = 0.1

FastSLAM Key Steps

1. **Extend the past posterior by sampling a new pose for each sample:**

$$\mathbf{x}_t^{[k]} \sim p(\mathbf{x}_t | \mathbf{x}_{t-1}^{[k]}, \mathbf{u}_t)$$

2. **Compute particle weight based on observation weighted by covariance (Mahalanobis Distance):**

$$w^{[k]} = \frac{1}{\sqrt{2\pi Q}} e^{-\frac{(\mathbf{z}_t - \hat{\mathbf{z}}^{[k]})^\top (\mathbf{z}_t - \hat{\mathbf{z}}^{[k]})}{2Q}}$$

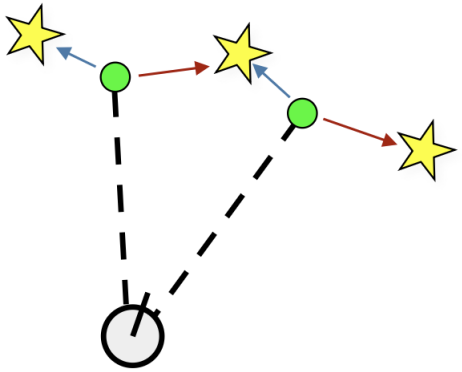
Here,

1. $Q = Q_t + H\Sigma_{j,t-1}^{[k]} H^\top$ is the measurement covariance obtained from measurement noise and landmark uncertainty (derivation: See appendix),
2. $\mathbf{z}_t - \hat{\mathbf{z}}^{[k]}$ represents the difference between the vector $\hat{\mathbf{z}}^{[k]}$ of all features observed at time t, and the vector $\mathbf{z}_t$ of their corresponding map features in particle k.

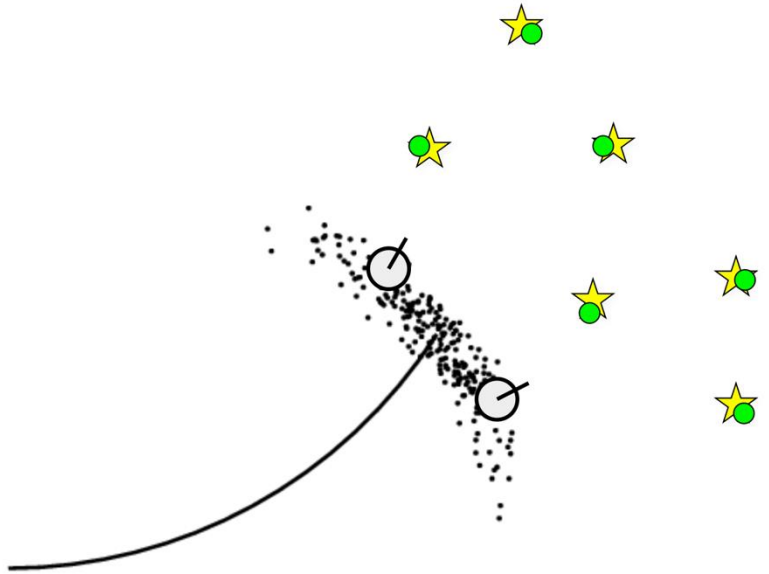
3. **Update belief of observed landmarks using the EKF formula.**
4. **Resample according to particle weights $w^{[k]}$.**

Data Association Problem

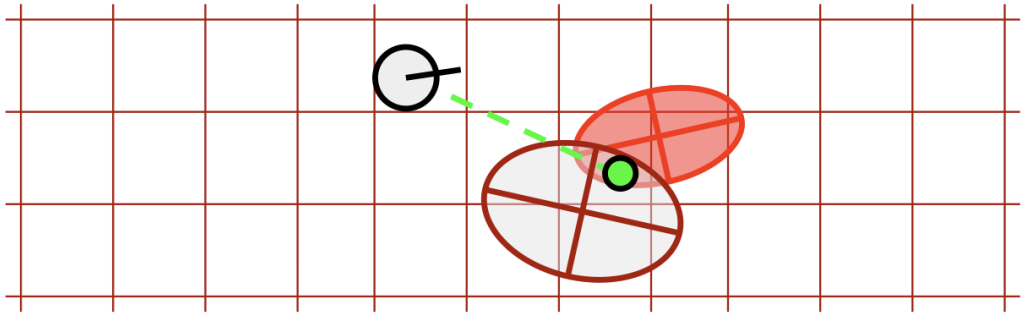
- Which observation belongs to which landmark?
- More than one possible association.
- Potential associations depend on the pose of the robot.



- In FastSLAM, decisions are made on a per-particle basis.
- Robot pose error is factored out of data association decisions.



Per-Particle Data Association

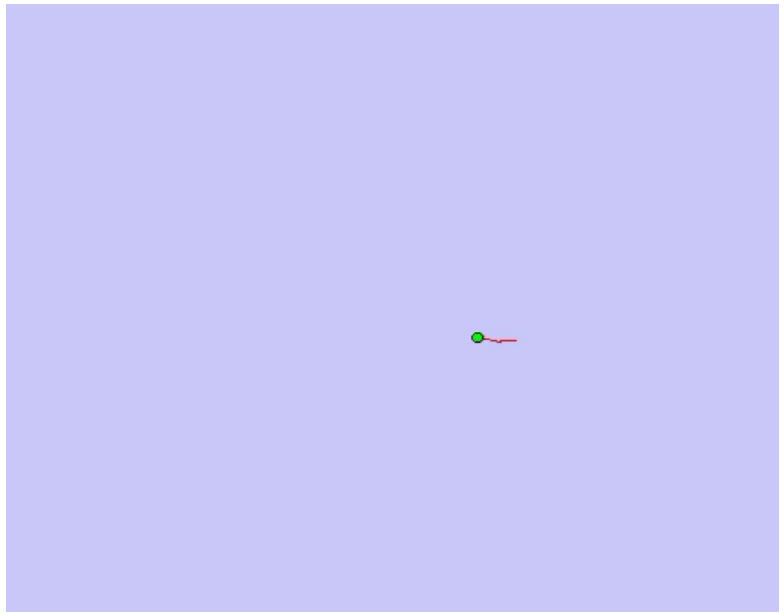


Was the observation generated by the **red** or by the **brown** landmark?

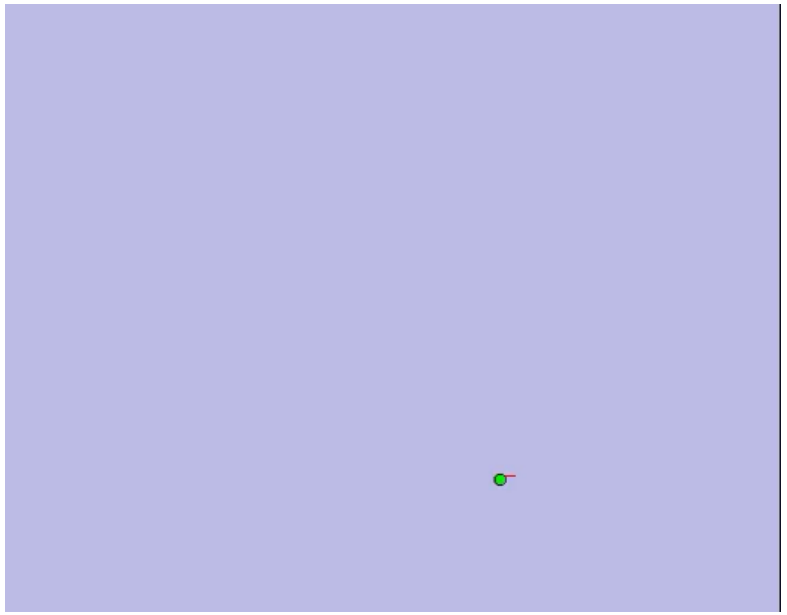
P(observation|red)=0.3
P(observation|brown)=0.7

- **Two options for per-particle data association:**
  - Pick the most probable match
  - Pick a random association weighted by the observation likelihoods
- **If the probability for an assignment is too low, generate a new landmark (in all particles!)**
- **This is a big advantage of FastSLAM over EKF SLAM:**
  - Inherent multi-modal belief
  - Pose error is factored out of data association decisions
  - Simple but effective data association

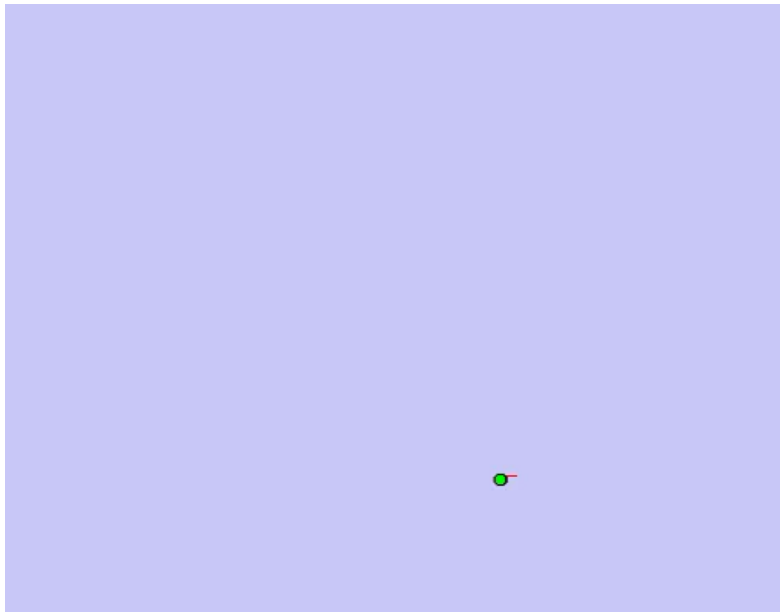
FastSLAM Example: Robot running in the Intel Research Lab



Raw Odometry



Odometry improved by
frame to frame scan matching



F2F scan matching
+ FastSLAM

FastSLAM Example: Robot running in the Bruceton Research Mine (link)

Close to worst case example:

- **Very long loops**
- **Very high self-similarity**



FastSLAM 1.0 Summary

- **Use a particle filter to model the belief**
- **Factors the SLAM posterior into low-dimensional estimation problems**
- **Model only the robot's path by sampling**
- **Compute the landmarks given the path**
- **Per-particle data association**
- **No robot pose uncertainty in the per-particle data association**
- **Complexity (N: number of particles, M: number of map features):**
  - Update robot particles based on the control: $O(N)$
  - Incorporate an observation into the Kalman filters: $O(N)$
  - Resample particle set: $O(NM)$
  - Overall: $O(NM)$
- **Compare with $O(M^2)$ for EKF-SLAM – that's where the name FastSLAM comes from!**

Using smart search and data structures this can even be reduced to $O(N \log M)$ – see [2]!

FastSLAM 1.0 → 2.0

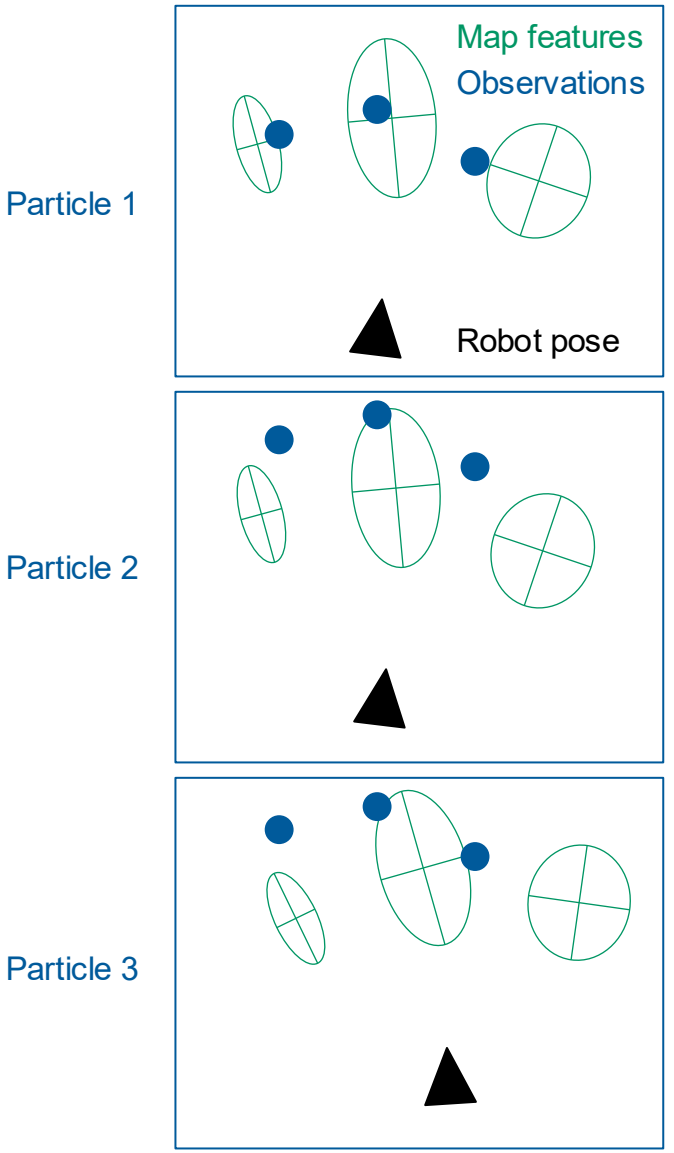
- FastSLAM 1.0 uses the motion model as the proposal distribution: $x_t^{[k]} \sim p(x_t | x_{t-1}^{[k]}, u_t).$
- Can we find a better distribution to sample from?

FastSLAM 1.0 → 2.0

- FastSLAM 1.0 uses the motion model as the proposal distribution: $x_t^{[k]} \sim p(x_t | x_{t-1}^{[k]}, u_t)$.
- Can we find a better distribution to sample from?
- Yes – we have a lot of landmark observation data that is completely unused here!
  (See the "Scan to Scan matching" in the Intel Research Lab example)
- FastSLAM 2.0 considers also the measurements during sampling.
- This is especially useful if we have an accurate sensor (compared to motion noise).

FastSLAM 1.0 → 2.0

- **FastSLAM 2.0 samples from** $x_t^{[k]} \sim p(x_t | x_{1:t-1}^{[k]}, u_{1:t}, z_{1:t}).$
- **I.e., it takes into account the sensor data.**
- **For example,**
  - multiply weights with average Mahalanobis distance of map features to observations (*matching criterion)*
  - use least-squares method to optimize position parameters based on map/observation matches, then weigh based on some distance measure of the particle to that "optimal" position
  - ...
- **This results in a more peaked proposal distribution.**
- **Less particles required, more robust and accurate**
- **But more complex...**



FastSLAM Summary

- **Particle filter-based SLAM**
- **Rao-Blackwellization: Model the robot's path by sampling, and compute the landmarks given the poses**
- **Allows for simple per-particle association, inherently addressing multi-modality and ambiguities**
- **FastSLAM 1.0 and 2.0 differ in the proposal distribution**
- **Complexity $O(NM)$ with naive data structures, $O(N \log M)$ with smarter setup**
- **Scales well (>1,000,000 features no problem)**
- **Much stronger than classical EKF approach.**

- **Previous lectures:**
  - Mathematical details of Particle Filters
  - Joint Probability Data Association Filters
  - JPDAs and Particle Filters
  - Simultaneous Localization and Mapping (based on Kalman Filters)
- **This week:**
  - Simultaneous Localization and Mapping (based on Particle Filters)
  - Overview over features for SLAM
  - Occupancy Grids and Grid-based SLAM
- **Upcoming:**
  - (DNN-based) early fusion approaches.

Appendix: Derivation of FastSLAM Importance Weight

Derivation of the FastSLAM Importance Weight

- **Based on the importance sampling principle:** $w^{[k]} = \frac{\text{target}(x^{[k]})}{\text{proposal}(x^{[k]})}$
- **Target distribution:** $p(x_{1:t}|z_{1:t}, u_{1:t})$
- **Proposal distribution:** $p(x_{1:t}|z_{1:t-1}, u_{1:t})$
- **Proposal is used step-by-step:**

$$p(x_{1:t}|z_{1:t-1}, u_{1:t}) = \underbrace{p(x_t|x_{t-1}, u_t)}_{\text{Motion}} \cdot \underbrace{p(x_{1:t-1}|z_{1:t-1}, u_{1:t-1})}_{\text{Last step's target}}$$

Derivation of the Importance Weight

$$w^{[k]} = \frac{\text{target}(x^{[k]})}{\text{proposal}(x^{[k]})}$$

$$= \frac{p(x_{1:t}^{[k]}|z_{1:t}, u_{1:t})}{p(x_t^{[k]}|x_{t-1}, u_t) \cdot p(x_{1:t-1}^{[k]}|z_{1:t-1}, u_{1:t-1})}$$

Target distribution: $p(x_{1:t}|z_{1:t}, u_{1:t})$

Proposal distribution: $p(x_{1:t}|z_{1:t-1}, u_{1:t})$

Proposal is used step-by-step:

$$p(x_{1:t}|z_{1:t-1}, u_{1:t}) = \underbrace{p(x_t|x_{t-1}, u_t)}_{\text{Motion}} \cdot \underbrace{p(x_{1:t-1}|z_{1:t-1}, u_{1:t-1})}_{\text{Last step's target}}$$

Derivation of the Importance Weight

$$
\begin{aligned}
w^{[k]} &= \frac{\text{target}(x^{[k]})}{\text{proposal}(x^{[k]})} \\
&= \frac{p(x_{1:t}^{[k]} | z_{1:t}, u_{1:t})}{p(x_t^{[k]} | x_{t-1}, u_t) \cdot p(x_{1:t-1}^{[k]} | z_{1:t-1}, u_{1:t-1})} \\
&= \frac{\eta \cdot p(z_t | x_{1:t}^{[k]}, z_{1:t-1}) \cdot p(x_t | x_{t-1}^{[k]}, u_t) \cdot p(x_{1:t-1}^{[k]} | z_{1:t-1}, u_{1:t-1})}{p(x_t^{[k]} | x_{t-1}, u_t) \cdot p(x_{1:t-1}^{[k]} | z_{1:t-1}, u_{1:t-1})}
\end{aligned}
$$

Bayes Rule
+ factorization

Derivation of the Importance Weight

$$
\begin{aligned}
w^{[k]} &= \frac{\text{target}(x^{[k]})}{\text{proposal}(x^{[k]})} \\
&= \frac{p(x_{1:t}^{[k]}|z_{1:t}, u_{1:t})}{p(x_t^{[k]}|x_{t-1}, u_t) \cdot p(x_{1:t-1}^{[k]}|z_{1:t-1}, u_{1:t-1})} \\
&= \frac{\eta \cdot p(z_t|x_{1:t}^{[k]}, z_{1:t-1}) \cdot \cancel{p(x_t|x_{t-1}^{[k]}, u_t)} \cdot \cancel{p(x_{1:t-1}^{[k]}|z_{1:t-1}, u_{1:t-1})}}{\cancel{p(x_t^{[k]}|x_{t-1}, u_t)} \cdot \cancel{p(x_{1:t-1}^{[k]}|z_{1:t-1}, u_{1:t-1})}} \\
&= \eta \cdot p(z_t|x_{1:t}^{[k]}, z_{1:t-1}).
\end{aligned}
$$

Derivation of the Importance Weight

- Integrating over the pose of the observed landmark leads to

$$
\begin{aligned}
w^{[k]} &= \eta \cdot p(z_t | x_{1:t}^{[k]}, z_{1:t-1}) \\
&= \eta \int_{l_j} p(z_t | x_{1:t}^{[k]}, z_{1:t-1}, l_j) \cdot p(l_j | x_{1:t}^{[k]}, z_{1:t-1}) dl_j \\
&= \eta \int_{l_j} \underbrace{p(z_t | x_t^{[k]}, l_j)}_{\mathcal{N}(\hat{z}^{[k]}, Q_t)} \cdot \underbrace{p(l_j | x_{1:t-1}^{[k]}, z_{1:t-1})}_{\mathcal{N}(\mu_{j,t-1}^{[k]}, \Sigma_{j,t-1}^{[k]})} dl_j
\end{aligned}
$$

$$
Q = Q_t + H \Sigma_{j,t-1}^{[k]} H^\top
$$

Literature / Links

1. A. Doucet &al., ***Rao-Blackwellised Particle Filtering for Dynamic Bayesian Networks***, https://arxiv.org/pdf/1301.3853
2. M. Montemerlo &al., ***FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem***, https://robots.stanford.edu/papers/montemerlo.fastslam-tr.pdf
3. D. Lowe: ***Distinctive Image Features from Scale-Invariant Keypoints,*** https://link.springer.com/article/10.1023/B:VISI.0000029664.99615.94
4. E. Rublee & al.: ***ORB: an efficient alternative to SIFT or SURF,*** https://www.researchgate.net/publication/221111151_ORB_an_efficient_alternative_to_SIFT_or_SURF



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!